

ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Η/Υ



**ΠΟΛΥΤΕΧΝΕΙΟ
ΚΡΗΤΗΣ /
TECHNICAL
UNIVERSITY
OF CRETE**

ΠΛΗ311: Αναφορά Τεχνητή Νοημοσύνη

2^η Εργασία:

ΑΙ με χρήση MINIMAX και MCTS για το παιχνίδι TUC-Chess

Ομάδα 140 :

Σκλάβος Παναγιώτης 2018030170

Διδάσκων:

Γεώργιος Χαλκιαδάκης

Επιβλέποντες:

Χρυσομάλλης Ιάσων

Ορφανουδάκης Σταύρος

1. Εισαγωγή

Στα πλαίσια της 2^{ης} εργασίας του μαθήματος, κληθήκαμε να υλοποιήσουμε έναν πράκτορα ο οποίος θα παίζει το παιχνίδι TUC-Chess. Το παιχνίδι αυτό αποτελεί μια παραλλαγή του σκακιού όπως το γνωρίζουμε, με ορισμένες αλλαγές στους κανονισμούς του. Το πρόγραμμα που υλοποιήσαμε ενσαρκώνει έναν παίκτη που προσπαθεί να κερδίσει στο παιχνίδι αυτό, με την χρήση 2 ευρέως γνωστών αλγορίθμων: Minimax, και Monte Carlo Tree Search. Ο τρόπος με τον οποίο εργάζονται οι δύο αλγόριθμοι είναι λίγο διαφορετικός, ωστόσο και οι δύο εργάζονται με γνώμονα την εύρεση της καλύτερης κίνησης για κάποιο δεδομένο Board, δηλαδή την εύρεση της κίνησης που θα οδηγήσει σε κάποιο θεμιτό για εμάς αποτέλεσμα. Σκοπός δηλαδή είναι πραγματοποιούμε αναζήτηση υπό αντιπαλότητα της καλύτερης κίνησης ώσπου να καταφέρουμε να νικήσουμε τον ανταγωνιστή μας.

2. Υλοποίηση Αλγορίθμων

2.1 Αλγόριθμος Minimax

Ο αλγόριθμος Minimax χρησιμοποιείται κατά κόρον σε Constant Sum ή απλούστερα Zero Sum παιχνίδια, όπως η Τρίλιζα και το Σκάκι, αποφέροντας πολύ αξιόπιστα και χρήσιμα αποτελέσματα. Η λογική πίσω από την λειτουργία του είναι να αξιολογεί όλα τα πιθανά σενάρια που απορρέουν από μια δεδομένη κατάσταση του παιχνιδιού αξιοποιώντας μια συνάρτησης αξιολόγησης που προσδιορίζει πόσο καλή είναι η κατάσταση αυτή για τον παίκτη μας. Η αναζήτηση αυτή μπορεί να γίνει για οποιοδήποτε αριθμό γύρων μετά την σειρά μας, αυξάνοντας ωστόσο των αριθμό των καταστάσεων που θα πρέπει να ελεγχθούν. Ο αλγόριθμος αυτός αξιολογεί τις καταστάσεις σε κάποιο συγκεκριμένο βάθος επιλέγει την κίνηση που είναι καλύτερη για τον παίκτη ο οποίος παίζει. Με άλλα λόγια, την κίνηση με μέγιστο κέρδος όταν αξιολογεί από την οπτική του παίκτη μας και την κίνηση με ελάχιστο κέρδος όταν αξιολογεί από την οπτική του αντιπάλου.

Στην φάση αυτή θα αναλυθεί όλη η μεθοδολογία της υλοποίησής μας. Αρχικά για την επιλογή της καλύτερης κίνησης που προκύπτει από τον αλγόριθμο, καλείται η συνάρτηση *selectActionMinimax ()* του World η οποία δημιουργεί ένα BoardState. Ένα στιγμιότυπο της κλάσης αυτής διατηρεί πληροφορία για το ποια είναι η τρέχουσα κατάσταση του board καθώς και ποιο είναι το score που επικρατεί στο παιχνίδι. Το *World* που δημιουργείται στο *World* περιγράφει όλη την πληροφορία του παιχνιδιού την τρέχουσα στιγμή και δίνεται ως όρισμα στην *MinimaxExec()* η οποία εφαρμόζει την λογική του Minimax με αναδρομικό τρόπο για να βρει την καλύτερη δυνατή κίνηση για την δοθείσα κατάσταση.

Συγκεκριμένα όντας αναδρομική συνάρτηση ελέγχουμε πρώτα τις τερματικές συνθήκες:

- ❖ Υπάρχει χρόνος ακόμη
- ❖ Έχουμε διανύσει το μέγιστο βάθος
- ❖ Έχουμε φθάσει σε Τερματική κατάσταση παιχνιδιού

Αν κάποια από τις προαναφερθείσες συνθήκες είναι αληθής σταματάμε την αναζήτηση και επιστρέφουμε την αξιολόγηση της κατάστασης που βρισκόμαστε.

Αν όμως η έρευνα συνεχίζεται ελέγχουμε αν στην κατάσταση που βρισκόμαστε υπολογίζουμε την κίνηση που θα πραγματοποιήσει ο παίκτης μας- MAX ή ο αντίπαλος- MIN. Και στις δύο άνωθεν περιπτώσεις η νοοτροπία είναι ίδια. Πραγματοποιούμε κάθε μια από τις διαθέσιμες κινήσεις του παίκτη δημιουργώντας μια νέα προσωρινή κατάσταση στο Board και το κάνουμε αυτό ώσπου να ικανοποιηθεί κάποια τερματική συνθήκη της αναδρομής. Όταν αυτό συμβεί ανανεώνουμε την καλύτερη κίνηση για κάθε βάθος αν η κίνηση που μας οδήγησε στην κατάσταση αυτή θεωρείται ότι επέφερε καλύτερο αποτέλεσμα.

Κατά την πραγματοποίηση της κίνησης ανανεώνεται επίσης και το score των δύο παικτών αν η κίνηση οδηγήσει σε αλλαγή του. Με τον τρόπο αυτό μπορούμε να έχουμε μια ακριβή εικόνα του τρέχοντος score για όταν θα χρειαστεί να αξιολογήσουμε πόσο θεμιτή θα είναι για εμάς μια κατάσταση. Αξίζει να σημειωθεί ότι τα νέα BoardStates που δημιουργούνται αποτελούν deepCopies του αρχικού στο οποίο γίνεται η κίνηση. Αυτό συμβαίνει με σκοπό να μην χάνεται η πληροφορία των προηγούμενων boards και συνεπώς να μην απαιτείται κάποιο undo της κάθε κίνησης μετά την κλήση της αναδρομικής συνάρτησης.

Έτσι λοιπόν σταδιακά για κάθε BoardState επιλέγεται το καλύτερο παιδί που μεγιστοποιεί τις δυνατότητές του να κερδίσει ώσπου επιλέγεται και για το αρχικό, που ήταν και το ζητούμενο. Έτσι επιλέγεται η κίνηση, που δεδομένου βέλτιστου παιχνιδιού του αντιπάλου, επιφέρει την πιο επικερδή κατάσταση για τον παίκτη μας.

2.2 α-β Prunning

Το α-β Prunning αποτελεί μια τεχνική για αποκοπή καταστάσεων κατά την αναζήτηση του Minimax καθώς το πλήθος τους αυξάνεται εκθετικά ως προς το βάθος, γεγονός που γεννά την ανάγκη για εύρεση τρόπων για εξερεύνηση λιγότερων εξ αυτών χωρίς να διακινδυνεύεται η αποδοτικότητά του. Μιας και η αναζήτηση στον Minimax γίνεται σαν DFS μέθοδος θα μπορούσαμε, αν το δέντρο αναζήτησης είναι καλά στοιχισμένο, με την μέθοδο του a-b pruning αποκόπταμε πολύ μεγάλα υποδέντρα αποφάσεων τα οποία με βεβαιότητα δεν μπορούν να αποφέρουν καλύτερες κινήσεις από τις ήδη ελεγμένες για τον παίκτη μας.

Αναλυτικότερα το α αντιπροσωπεύει την καλύτερη τιμή αξιολόγησης για τον για τον παίκτη μας (μέγιστη) που έχει βρεθεί οπουδήποτε κατά μήκος της διαδρομής, ενώ το β αντιπροσωπεύει την καλύτερη τιμή για τον αντίπαλο (ελάχιστη) κατά αντίστοιχο τρόπο. Συνεπώς όταν κατά την εκτέλεση του Minimax είμαστε σε περίπτωση που παίζει ο Max, αν κατά την εκτέλεση βρεθεί κάποιο evaluation μεγαλύτερο του α τότε ανανεώνεται το α και επίσης πραγματοποιείται ο εξής έλεγχος: Το β να είναι μικρότερο ίσο με το α. Αν αυτό ισχύει σημαίνει ότι σε κάποιο κόμβο σε προηγούμενο στάδιο της εξερεύνησης είχαμε βρει κάποιο κόμβο με καλύτερη τιμή καθώς το β προέρχεται από την κατάσταση πατέρα, και αποτελεί την καλύτερη τιμή έως τώρα για τον πατέρα. Αν λοιπόν το α που είναι η καλύτερη τιμή έως τώρα για τον κόμβο(MAX) έχει κάποια τιμή μεγαλύτερη του β αυτό σημαίνει ότι οποιοσδήποτε άλλος

κόμβος και να επιλεγεί ως καλύτερος ο κόμβος του MAX θα έχει τιμή $eval \geq a > b$. Άρα ο κόμβος max αυτό δεν χρειάζεται να εξερευνηθεί άλλο καθώς με βεβαιότητα ο MIN πατέρας δεν θα τον επέλεγε και θα προτιμούσε τον κόμβο που του έδωσε την τιμή του β εξαρχής.

Όσον αναφορά την στοίχιση των κόμβων που αναφέρθηκε προηγουμένως, είναι δυνατόν να αυξήσουμε επιπλέον την αποτελεσματικότητα του α-β pruning αν οι κόμβοι πριν την εξερεύνηση ήταν στοιχισμένοι έτσι ώστε αυτοί με καλύτερη πιθανότητα να είναι βέλτιστοι να εξερευνούνται πρώτοι. Με τον τρόπο αυτό αν βρίσκαμε γρήγορα καλές τιμές για α και β θα μπορούσαμε να αποκόψουμε την πλειοψηφία του δέντρου με τεράστια οφέλη σε απόδοση. Αντίθετα αν είχαμε την αντίστροφη στοίχιση δεν θα παρατηρούνταν σχεδόν καμία αποκοπή κατά την αναζήτηση.

2.3 Ευρετικές, Cuttof Search και Βελτιώσεις

Τόσο για την ευρετική που εφαρμόστηκε για την αξιολόγηση των καταστάσεων στον Minimax όσο και για την λογική της αποκοπής που αξιοποιήθηκε στο α-β pruning έγινε χρήση κάποιας συνάρτησης αξιολόγησης η οποία κατέστησε εφικτό να κρίνονται κάποιες καταστάσεις περισσότερο ή λιγότερο θεμιτές.

Στην παραδοτέα υλοποίηση η συνάρτηση αξιολόγησης στην οποία δομήθηκαν οι παραπάνω αποφάσεις, δεν είναι άλλη από την προτεινόμενη από την εκφώνηση με κάποιες μικρές τροποποιήσεις.

$$V(\text{state}) = |\text{my (points + pieces)}| - |\text{opponent's (points + pieces)}|$$

Οι πόντοι του κάθε παίκτη βρίσκονται απόθηκευμένοι στο StateBoard που αξιολογείται ενώ η αξία που αποδίδεται στα πόνια κατά την αξιολόγησή τους ως pieces είναι ίση με τους πόντους που αποδίδουν σε εκείνον που τα αιχμαλωτίζει.

Η συνάρτηση είναι ικανοποιητική καθώς προσπαθεί κάθε κίνηση να επιφέρει αφενός κερδοφόρο score και αφετέρου προβάδισμα στα πόνια που έχουμε πάνω στο Board (στρατιώτες για την συνέχεια). Ωστόσο είναι αρκετά περιορισμένη καθώς δεν υπάρχει κάποια δυναμικότητα στον τρόπο με τον οποίο επιλέγονται οι καταστάσεις αναλόγως την φάση του παιχνιδιού ή κάποια εξυπνότερη παρατήρηση που να οδηγεί σε καλύτερες στρατηγικές. Μια επιπλέον συνθήκη που ελέγχεται ωστόσο είναι αν κάποιος βασιλιάς απειλείται. Στην περίπτωση αυτή η κατάσταση αυτή είναι θεμιτή οπότε λαμβάνει μεγάλο reward αν ο παίκτης μας (maximizing player) βρίσκεται έως και 7 πόντους πίσω συνεπώς με την κατάκτηση του βασιλιά κερδίζει την παρτίδα, ενώ αν με την αιχμαλώτιση του βασιλιά χάνει η κίνηση αυτή αποφεύγεται.

Η ίδια συνάρτηση χρησιμοποιείται και για την αξιολόγηση των καταστάσεων κατά το α-β pruning καθορίζοντας με τον τρόπο αυτό ποιοι κόμβοι αξίζει και ποιοι όχι να εξετασθούν περαιτέρω.

Μια άλλη τροποποίηση που υλοποιήθηκε για την βελτίωση της απόδοσης του α-β pruning αυτή τη φορά, είναι να γίνεται μια σύντομη στοίχιση των κινήσεων προτού ελεγχθούν, τοποθετώντας πρώτες τις αιχμαλωτίσεις, καθώς είναι περισσότερο πιθανόν να οδηγήσουν σε νικητήριες καταστάσεις.

Προφανώς και υπάρχουν ποικίλοι τρόποι με τους οποίους η απόδοση θα μπορούσε να βελτιωθεί περαιτέρω οι οποίοι θα σχολιαστούν στην ενότητα 7.

2.4 Αλγόριθμος Monte Carlo Tree Search

Ο Αλγόριθμος αυτός σε αντίθεση με τον Minimax μπορεί με μια πιο ευρεία αντιμετώπιση του προβλήματος να υπολογίζει σε κάθε στιγμή την καλύτερη δυνατή κίνηση για τον παίκτη μας. Αυτό σημαίνει ότι για την χρήση του αλγορίθμου δεν είναι απαραίτητο να εφαρμόζουμε και κάποια τακτική αξιολόγησης όπως είδαμε στον Minimax. Εμβαθύνοντας λίγο παραπάνω, ο Monte Carlo βασίζεται κυρίως σε αξιοποίηση στατιστικών για την λήψη αποφάσεων, και όχι σε περίπλοκους υπολογισμούς και ευρετικές.

Ο τρόπος με τον οποίο λειτουργεί ο αλγόριθμος Monte Carlo είναι να δοκιμάζει με χιλιάδες τυχαίες προσομοιώσεις από την οποίες λαμβάνει κάποια στατιστικά για τις πιθανότητες να νικής ακολουθώντας ένα μονοπάτι. Ο τρόπος με τον οποίο γίνονται τα παραπάνω περικλύεται στα εξής 4 στάδια:

- Select: Το στάδιο αυτό στην υλοποίηση που παραδόθηκε γίνεται εφαρμόζοντας την UCB1 στρατηγική που είναι η στρατηγική που προκύπτει αν στη $UCT = V_i + 2 C \sqrt{\frac{\ln N}{n_i}}$, αποδώσουμε στο C την τιμή $\frac{1}{\sqrt{2}}$ δεδομένου ότι οι τιμές που λαμβάνουμε από το V_i είναι στο εύρος $-1 \leq 0 \leq 1$. Στην άνωθεν αναπαράσταση το V_i αποτελεί το μέσο κέρδος της κατάστασης, το N: το πλήθος των συνολικών επισκέψεων και το n_i το πλήθος των επισκέψεων της συγκεκριμένης κατάστασης. Με την χρήση λοιπόν του κανόνα αυτού επιλέγουμε κάθε φορά τον κόμβο που έχει την καλύτερη UCB1 τιμή. Κατά το πρώτο iteration ενός κόμβου είναι προφανές ότι η τιμή του UCB = inf καθώς το $n_i = 0$.
- Expand: Στο στάδιο αυτό ερχόμαστε μόνο στην περίπτωση που ο κόμβος που έδωσε η select έχει $n_i = 0$. Αυτό σημαίνει ότι έχουμε τρέξει τουλάχιστον ένα iteration από όλα τα nodes. Αυτό που συμβαίνει λοιπόν είναι, αν έχουμε διαθέσιμες κινήσεις στην κατάσταση που βρισκόμαστε και ο κόμβος μας δεν είναι τερματικός για το παιχνίδι τότε, εφαρμόζουμε κάθε μια από τις κινήσεις αυτές και έτσι δημιουργούμε όλα τα παιδιά Boards που απορρέουν με μία νόμιμη κίνηση από το τρέχον μας node. Αν λοιπόν πραγματοποιηθεί το expand, ως current κόμβος για την αναζήτηση μας ορίζεται ένα τυχαίο παιδί του κόμβου που επεκτάθηκε. Αλλιώς πάμε στην επόμενη φάση με τον κόμβο που έδωσε η select.
- Rollout: Εδώ τώρα, γίνεται ένα random simulation από την κατάσταση που είχαμε ως current κόμβο μέχρι κάποια τερματική. Όταν φθάσουμε σε τερματική μπορούμε να την

αξιολογήσουμε επιστρέφοντας 1 αν είναι αν επιφέρει νίκη, -1 αν επιφέρει ήττα ή 0 σε ισοπαλία.

- Backpropagate: Στο τελικό αυτό στάδιο, ξεκινώντας από τον κόμβο που αρχίσαμε την προσομοίωση, αρχίζουμε να ανανεώνουμε τις επισκέψεις και το συνολικό reward στην διαδρομή από τον κόμβο έως το root ώστε να ενημερωθούν όλοι για το αποτέλεσμα του rollout.

4. Σχολιασμός Υλοποιήσεων Και Πειραματισμών

Προτού απεικονισθούν τα αποτελέσματα των διάφορων προσομοιώσεων αναμετρήσεων μεταξύ των αλγορίθμων, καλό θα ήταν να γίνει μια σύντομη μνεία στις συνθήκες των πειραματισμών και τις αποφάσεις προσπάθειες που επιχειρήθηκαν. Αρχικά, για τον Minimax δοκιμάστηκαν πολύ απλές ευρετικές σε πρώτη φάση, οι οποίες απλώς επέστρεφαν την διαφορά στο Score προκειμένου να γίνει ορατή η βελτίωση στην απόδοση με την ελαφρώς καλύτερη ευρετική που εφαρμόστηκε τελικά. Και η διαφορά ήταν αρκετά μεγάλη. Μια επέκταση την ευρετικής αυτής που δοκιμάστηκε, είναι, σε περίπτωση που στο Board που αξιολογείται καταλήγει σε τερματική κατάσταση, τότε να επιστρέφεται κάποιο μεγάλο score αν ο παίκτης μας νικάει ή πολύ μικρό αν χάνει. Ωστόσο λόγω της μη σαφούς βελτίωσης στην συμπεριφορά, εν τέλει τοποθετήθηκε σε σχόλια.

Επιπλέον αξίζει να σημειωθεί ότι το Score που αντιλαμβάνεται ο αλγόριθμος του minimax και του α-β, όταν ξεκινάει κάθε αναζήτηση ανανεώνεται με το Score που υπάρχει στο πραγματικό Board καθώς περνιέται στο StateBoard που δίνεται ως όρισμα, και δεν αποτελεί μονάχα το προσδόκιμο. Έπειτα εσωτερικά προσμετρώνται οι αυξήσεις που απαιτούνται λόγω των αιχμαλωτίσεων με κινήσεις.

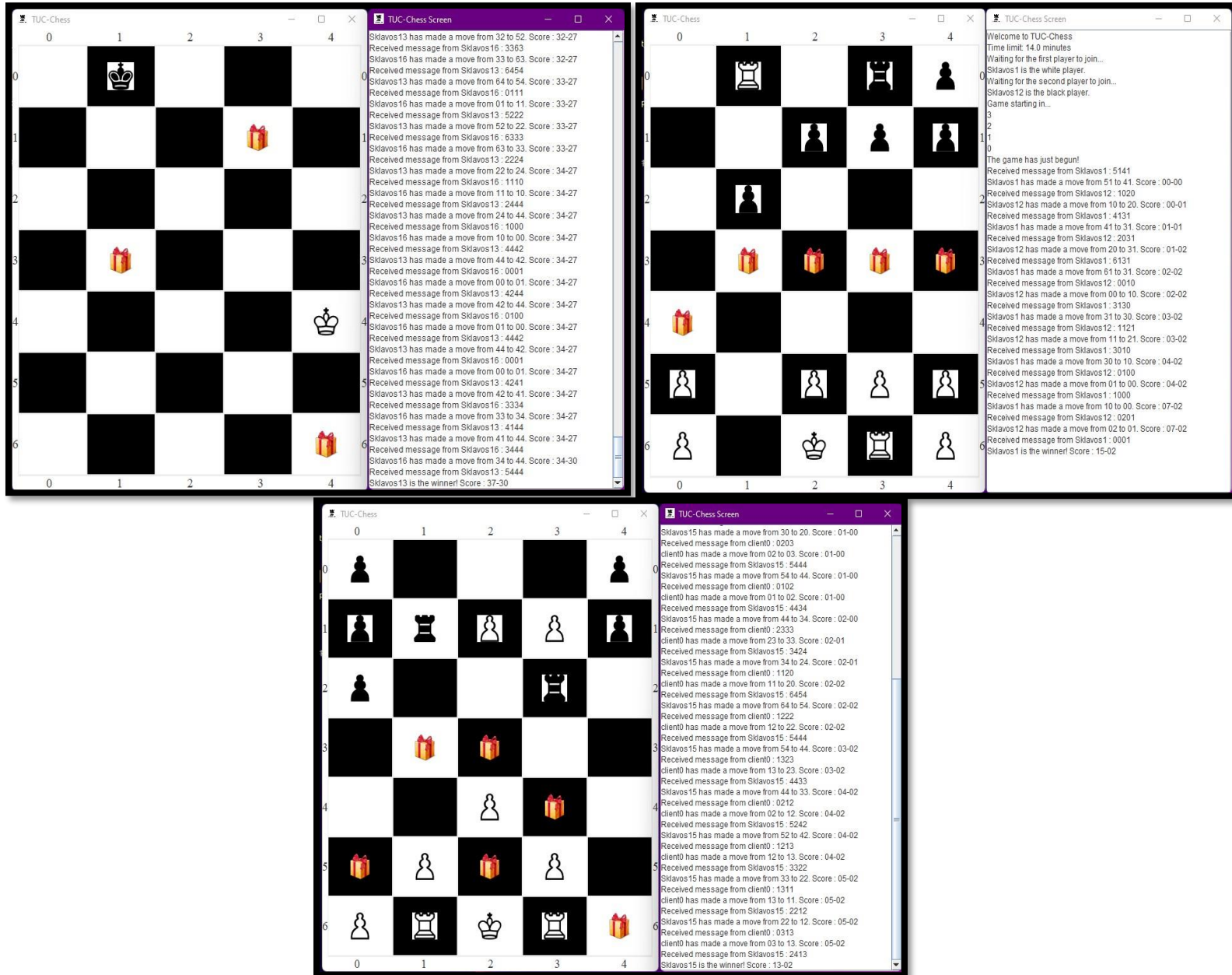
Επίσης πολυάριθμες συναρτήσεις που χρησιμοποιούνται τόσο σε MCTS όσο και σε Minimax τοποθετήθηκαν στην κλάση Search την οποία αμφότερες κάνουν extend.

Όσον αφορά την υλοποίηση του α-β αξίζει να σημειωθεί ότι υλοποιήθηκε συνάρτηση η οποία ταξινομεί τις κινήσεις ώστε να επιχειρούνται πρώτα οι κινήσεις αιχμαλωτίσεων καθώς είναι περισσότερο πιθανό να οδηγούν σε θεμιτό αποτέλεσμα. Όμως, λόγω έλλειψης αποτελεσματικότητας της ευρετικής η οποία δεν βελτιώθηκε αρκετά, ορισμένες φορές η χρήση της ταξινομημένης λίστας κινήσεων μπορεί να οδηγήσει σε σημαντικές αμυντικές αμέλειες που οδηγούν σε ήττα.

Τέλος Για τον Monte Carlo αλγόριθμο παρήχθη η κλάση SearchNode η οποία περιέχει σημαντική πληροφορία για την αναζήτηση που πραγματοποιείται όπως τα παιδιά κάθε κόμβου, τον πατέρα του, την κίνηση που οδήγησε σε εκείνον, κλπ. Επίσης για τον αριθμό C επιλέχθηκε η τιμή $\frac{1}{\sqrt{2}}$ καθώς με βάση την θεωρία όταν για τον αλγόριθμο MCTS δεν χρησιμοποιείται κάποια ευρετική για αξιολόγηση, άλλα το αποτέλεσμα νίκη (reward = 1), ισοπαλίας (reward = 0) ή ήττας (reward = -1).

5. Αποτελέσματα Πειραμάτων

Για αποφυγή πλεονασμών και επαναλήψεων θα παρασταθούν δειγματοληπτικά τα 3 σημαντικότερα αποτελέσματα που προέκυψαν από τις προσομοιώσεις. Συγκεκριμένα παρουσιάζεται η αναμέτρηση μεταξύ AB και Minimax, όπου ξεχωρίζει ο ισχυρότερος, έπειτα η αναμέτρηση μεταξύ MCTS και Minimax καθώς με την νίκη του Minimax υπονοείται το αποτέλεσμα της αναμέτρησης MCTS και α-β pruning. Τέλος αναπαρίσταται η αναμέτρηση του MCTS με τον random client ο οποίος όπως είναι αναμενόμενο, χάνει από τους υπόλοιπους.



- ❖ $\{\alpha\beta - \text{Minimax}\} = \{37-30\}$
- ❖ $\{\text{Minimax} - \text{MCTS}\} = \{13-02\}$
- ❖ $\{\text{MCTS} - \text{random}\} = \{15-02\}$
- ❖ $\{\text{MCTS} - \alpha\beta\} = \{06-22\}$
- ❖ $\{\alpha\beta - \text{Random}\} = \{21-01\}$
- ❖ $\{\text{Minimax} - \text{Random}\} = \{15-03\}$

6. Σχολιασμός Αποτελεσμάτων και Συμπεράσματα

6.1 Σύγκριση Minimax και Minimax με α - β pruning

Ορισμένες σημαντικές παρατηρήσεις προκύπτουν από την σύγκριση των 2 αλγορίθμων. Συγκεκριμένα στην υλοποίηση που πραγματοποιήσαμε μια ενδιαφέρουσα πληροφορία που αποσπάσαμε είναι η ακόλουθη. Μιας και στις δύο αναζητήσεις μία από τις κοινές συνθήκες για επιστροφή αξιολόγησης είναι να φθάσουμε σε μέγιστο βάθος, όταν το βάθος είναι αρκετά μικρό, οι δύο αλγόριθμοι πολλές φορές συμπεριφέρονται με τον ίδιο τρόπο. Αυτό συμβαίνει διότι και οι δύο προλαβαίνουν να φθάνουν στο μέγιστο βάθος κάθε φορά και να ολοκληρώνουν την αναζήτησή τους προτού τελειώσει ο χρόνος. Για να φανεί η ανωτερότητα του α - β θα πρέπει ο Minimax για δεδομένες παραμέτρους χρόνου και βάθους να μην προλαβαίνει να εξερευνήσει όλο το βάθος. Έτσι η βελτίωση του α - β pruning θα φθάνει πιο χαμηλά στο δέντρο αναζήτησης.

Μια ακόμη σημαντική παρατήρηση είναι ότι η χρήση μιας καλής ευρετική είναι πολύ σημαντική για να αναδειχθούν ακόμη περισσότερο οι δυνατότητες του α - β καθώς εντοπίζοντας με μεγαλύτερη ακρίβεια τις επικερδείς καταστάσεις θα μπορούν να αποκόπτονται περισσότεροι κόμβοι αναζήτησης.

Σε γενική ανάλυση ωστόσο ο α - β θα πρέπει σε κάθε περίπτωση να λειτουργεί καλύτερη ή το ίδιο αποδοτικά με τον Minimax καθώς και οι δύο οδηγούν στις ίδιες αποφάσεις, απλώς ο με το κλάδεμα α - β μπορούμε να οδηγούμαστε ταχύτερα σε αυτές.

6.2 Σύγκριση Monte Carlo με Minimax αλγορίθμους

Ο Monte Carlo παρόλο που έχει ορισμένα πολύ ισχυρά σημεία αποφάνθηκε ότι για τις ανάγκες του δικού μας παιχνιδιού υστερεί σε σχέση με τον Minimax. Συγκεκριμένα, ο MCTS αποτελεί έναν anytime αλγόριθμο, δηλαδή επιστρέφει την καλύτερη λύση έως οποιαδήποτε στιγμή του ζητηθεί. Δεν απαιτεί κάποια πολύπλοκη ευρετική που λαμβάνει υπόψιν παράγοντες που απαιτούν μεγάλη κατανόηση του παιχνιδιού, μπορεί να υπολογίζει λύση με ελάχιστη πληροφορία, καθώς μπορεί να αξιοποιηθεί και μόνο η κατάσταση νίκης στην τερματική που οδηγούν οι τυχαίες του επιλογές. Επιπλέον ο MCTS για τα αποτελέσματά του βασίζεται σημαντικά στην τυχαιότητα με αποτέλεσμα ορισμένες φορές να χάνει κάποιες καλές επιλογές. Επιπλέον η απόδοση του εξαρτάται σημαντικά από το χρόνο που θα του δοθεί και από το πλήθος των πειραμάτων που θα εκτελέσει για κάποια κατάσταση. Τέλος ο MCTS διαπρέπει σε παιχνίδια και καταστάσεις όπου το branching factor είναι πάρα πολύ μεγάλο για τον Minimax ώστε να ελέγχει όλες τις καταστάσεις έως κάποιο ικανοποιητικό βάθος, δηλαδή παιχνίδια όπου έχουν πάρα πολύ μεγάλο αριθμό επιτρεπτών μεταβάσεων.

Από την άλλη ο Minimax είναι περισσότερο κατάλληλος για το παιχνίδι TUC-Chess, τουλάχιστον για την απλή υλοποίηση χωρίς σημαντικές βελτιώσεις που παραδίδεται για την άσκηση αυτή. Αυτό συμβαίνει διότι το branching factor μας είναι ιδιαίτερα περιορισμένο (5-9 κινήσεις). Λόγω των λιγότερων καταστάσεων προς έλεγχο επίσης μπορεί αρκετά γρήγορα να λαμβάνει καλές και ασφαλείς αποφάσεις προβλέποντας τις αποφάσεις του αντιπάλου για ένα μικρό βάθος της τάξεως των 5 κινήσεων. Ο περιορισμός της τυχαιότητας που επιφέρει είναι

ιδιαίτερα σημαντικός καθώς κάθε κατάσταση μπορεί να αξιολογηθεί με αρκετά καλή ακρίβεια. Επίσης οι κανόνες του παιχνιδιού μας επιτρέπουν εύκολα να ορίσουμε κάποια ευρετική που θα βελτιώνει τις αποφάσεις μας.

Συνεπώς βλέπουμε ότι πολλά από τα δυνατά σημεία του Minimax είναι αρκετά χρήσιμα στην εύρεση βέλτιστης λύσης στο πρόβλημά μας. Αντιθέτως πολλά από τα καλά στοιχεία του MCTS φαίνεται να μην μας είναι απαραίτητα. Το γεγονός αυτός έχει ως αποτέλεσμα αυτό που είδαμε και από τις προσωμοιώσεις το οποίο είναι ότι παρόλο που προφανώς ο MCTS μπορεί να οδηγήσει σε αξιόλογες επιλογές που του δίνουν την νίκη απέναντι σε έναν εύκολο αντίπαλο, δεν είναι αρκετά ευφυής ώστε να υπερνικήσει τον αντίπαλο Minimax, ο οποίος φαίνεται να λαμβάνει στην πλειοψηφία του καλύτερες αποφάσεις.

7. Προτάσεις για Μελλοντικές Βελτιώσεις

Παρά τα όσα είπαμε παραπάνω και οι 2 αλγόριθμοι μπορούν να λάβουν σημαντικές βελτιώσεις οι οποίες θα αυξήσουν περαιτέρω την απόδοσή τους και θα τους καταστήσουν ακόμη καλύτερους για χρήση σε προβλήματα εύρεσης βέλτιστης κίνησης. Ενδεικτικά για τον Minimax:

- Υπαρξη προεπιλεγμένων moves σε HashMap. Τα HashMaps αυτά μπορούν να δημιουργούνται κατά την DFS κατάβασή μας στο δέντρο, κρατώντας βέλτιστες κινήσεις για τα αριστερότερα παιδιά τις οποίες μπορεί να χρειαστεί να χρησιμοποιήσουμε σε όμοιες καταστάσεις στο μέλλον. Ως κλειδί στο HashMap απλώς θα δίνεται το Board και θα επιστρέφεται κάποια βέλτιστη κίνηση αν έχει υπολογιστεί
- Αντίστοιχα θα μπορούσαν να εκμεταλλευτούν και οι Συμμετρίες του Board μειώνοντας σημαντικά τα evaluations του Board
- Χρήση BitBoards ώστε να μπορούν ταχύτερα να γίνονται έλεγχοι τερματικών καταστάσεων κλπ με bitwise operations. Με τον τρόπο αυτό θα μειωνόταν σημαντικά ο χρόνος που αφιερώνεται σε τέτοιους ελέγχους με την απλή υλοποίηση. Μάλιστα αυτοί οι έλεγχοι είναι και σημαντικοί παράγοντες που μειώνουν την ταχύτητα του Minimax
- Πολύ βασική βελτίωση για τον Minimax με αβ pruning είναι να γίνεται ένα presort των κινήσεων μέρος του οποίου έχει γίνει και στην υλοποίηση που παραδίδεται (Πρώτα οι κατακτήσεις). Συγκεκριμένα πρώτα τα Hashed moves, μετά οι αλληλουχίες capture που νικάμε έπειτα οι αλληλουχίες που επιφέρουν ίσες ανταλλαγές και έπειτα αμυντικές κινήσεις. Γενικά θα μπορούσε να ενταχθεί μεγάλη πληθώρα υποκατηγοριών.
- Επιπρόσθετες cutoff τεχνικές όπως Forward Pruning κλπ. (Δεν αναφέρονται περισσότερα μιας και έχουν ήδη σχολιασθεί στο μάθημα),
- Δεν θα μπορούσαμε να παραλείψουμε την χρήση καλύτερης ευρετικής, καθώς και τον έλεγχο για καταστάσεις που μπορούν να επιφέρουν άμεσα νίκη.

Και πολλές άλλες θα μπορούσαμε να αναφέρουμε καθώς είναι εφικτό με πολυάριθμους τρόπους να κάνουμε πιο έξυπνη και αποδοτική την αναζήτησή μας.

Monte Carlo:

Ενδεικτικά θα μπορούσε να βελτιωθεί χρησιμοποιώντας κάποια αξιόπιστη ευρετική για την αξιολόγηση καταστάσεων που μπορεί να μας επιδεκνύει με μεγαλύτερη ακρίβεια την αξία τους. Σίγουρα εξαιρετική άλλη βελτίωση θα ήταν ο συνδυασμός με άλλες μεθόδους ακόμη και τον Minimax ώστε να λαμβάνονται περισσότερο εύστοχες αποφάσεις. Τέλος όπως είδαμε και με το πρόγραμμα που κέρδισε τον παγκόσμιο Πρωταθλητή στο Go, θαύματα θα μπορούσαν να επιτευχθούν αν συνδυαστούν Deep Learning και Neural Networks τεχνικές για την βελτίωση της απόδοσης του αλγορίθμου.

8. Επίλογος

Συνοψίζοντας στα πλαίσια του μαθήματος καταφέραμε τελικά να δημιουργήσουμε 2 προγράμματα τα οποία αξιοποιώντας διαφορετικούς πόρους και τεχνικές μπορούν να οδηγήσουν σε αρκετά καλές και οφέλιμες κινήσεις για τον παίκτη μας. Οι γνώσεις που λήφθηκαν από την εργασία αυτή είναι ιδιαίτερα σημαντικές καθώς ο τομέας αυτός ακμάζει στις μέρες μας καθώς η αυτοματοποιημένη λήψη αποφάσεων που αποτελεί επέκταση του προβλήματος που κληθήκαμε να επιλύσουμε, αποτελεί έναν χώρο που αναμένεται να λάβει σημαντική ανάπτυξη με την δημιουργία πιο έξυπνων AI πρακτόρων. Το μόνο που απομένει είναι η μελλοντική ενασχόληση και η βελτίωση των βάσεων που θέσαμε με την εκπόνηση του Project.

9. Βιβλιογραφία

- <https://webdocs.cs.ualberta.ca/~hayward/396/jem/mcts.html>
- <https://www.baeldung.com/java-monte-carlo-tree-search>
- <https://levelup.gitconnected.com/improving-minimax-performance-fc82bc337dfd>
- https://www.chessprogramming.org/Move_Ordering
- <https://www.youtube.com/watch?v=l-hh51ncgDI&t=93s>
- <https://www.youtube.com/watch?v=-ivz8yJ4l4E>